

AGENTIC AI using LangGraph

Agentic AI:

Agentic AI is a type of AI that can take up a task or goal from user and then work towards completing it on its own with minimal human guidance

⇒ It plans, take actions, adapt to changes and seeks help only when necessary.

⇒ Generative AI is reactive (give response to what we ask) where Agentic AI is proactive (We give goal and it acts accordingly)

Key Characteristics of Agent AI

- ① Autonomous
- ② Goal Oriented
- ③ Planning
- ④ Reasoning
- ⑤ Adaptability
- ⑥ Context Awareness

① Autonomy:

Autonomy refers to AI system's ability to make decisions and take actions on its own to achieve a given goal without needing step-by-step human instructions

- It's proactive.
- Autonomy in multiple facets:
 - a. Execution
 - b. Decision Making
 - c. Tools usage

- Autonomy can be controlled
 - a. **Permission scope**: Limit what tools or actions the agent can perform independently

- b. **Human-in-the-loop (HITL)**: Insert checkpoints where human approval is required before continuing.

- c. **override controls**: Allow user to stop, pause or change the agent's behaviour at any time.

- d. **Policies**: Define hard rules or ethical boundaries the agent must follow.

2. Goal Oriented:

Being goal oriented means that the AI system operates with a persistent objective in mind and continuously direct its actions to achieve

that objective rather than just responding to isolated prompts.

- Goals act as a compass for autonomy.
- Goals can come with constraints
- Goals are stored in core memory
- Goals can be altered.

3. Planning:

Planning is the agent's ability to break down a high level goal into a structured sequence of actions or subgoals and decide the best path to achieve the desired outcome.

Step 1: Generating multiple candidate plans

Step 2: Evaluate each plan
According to:

- Efficiency
- Tool Availability
- Cost
- Risk
- Alignment

Step 3: Select the best plan with the help of

- Human-in-the-loop input
- A pre-programmed policy.

4. Reasoning,

Reasoning is the cognitive process through which an agentic AI system interprets information, draw conclusions and make decisions - both while planning and while executing in real time.

Reasoning during planning:

- **Goal Decomposition:** Break down abstract goals into concrete steps
- **Tool selection:** Decide which tool will be needed for which step
- **Resource estimation:** Estimate time, dependencies, risks

Reasoning during execution:

- Decision making
- HITL (Human-in-the-loop)
Handelling
- Error Handelling

5. Adaptability:

Adaptability is the agent's ability to modify its plans, strategies or actions in response to unexpected conditions - all while staying aligned with the goal.

- Failures (API fail)
- External feedback (less no. of applications)
- Changing goals midway

6. Context Awareness:

Context Awareness is the AI agent's ability to understand, retain and utilize relevant information from the ongoing task, past interactions, user preferences and environmental cues to make better decisions through a multi-step process

(i) Types of context:

- a. The original goal
- b. Progress till now + interaction history
- c. Environment state

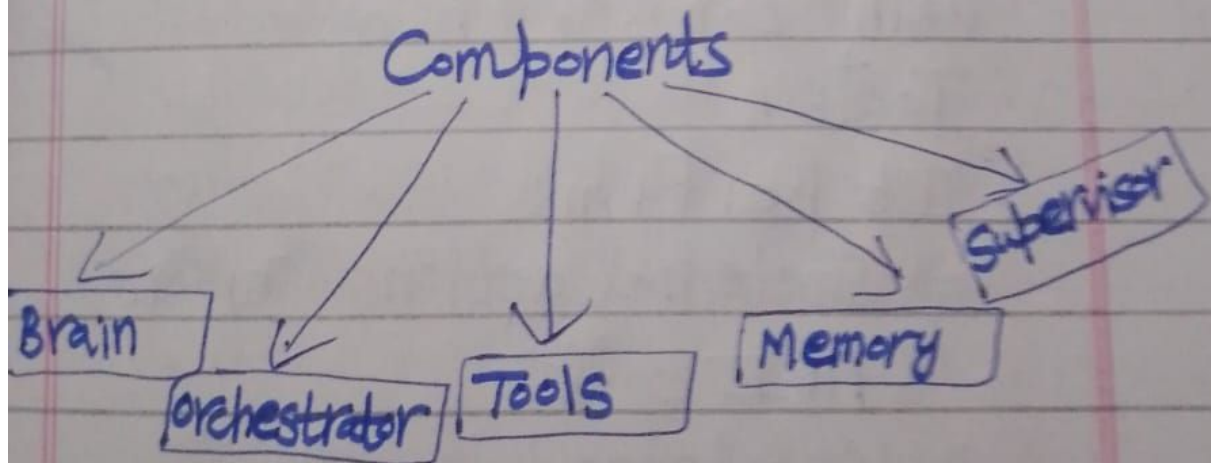
- d. Tool responses
- e. User specific preferences
- f. Policy or guardrails

(ii) Context Awareness is implemented through memory

Two types of memory:

- Short term memory
- Long term memory.

⇒ Components of Agentic AI



1. Brain:

- Usually LLM

It performs:

⇒ Goal Interpretation ⇒ Planning

⇒ Reasoning ⇒ Tool selection ⇒ Communication

2. Orchestrator

- It performs:

- ⇒ Task sequencing
- ⇒ Conditional routing
- ⇒ Retry logic
- ⇒ Looping and iteration
- ⇒ Delegation ⇒ Decided whether to hand off work to tools, LLM or humans.

- We build orchestrator using LangGraph etc.

3. Tools:

- It performs:

- ⇒ External actions: Perform API calls

- ⇒ Knowledge base access: Retrieve factual or domain specific information using RAG

4. Memory:

Short-Term Memory:

Maintains the active session's context - recent user messages, tool calls and immediate decisions.

Long-Term Memory:

~~Per~~^{sist} High level goals, user preferences and decisions across contexts.

State Tracking:

Monitors progress what's completed, what's pending

5. Supervisor

- Used for Human-in-the-loop (HITL)

It performs:

- ⇒ Approval Requests (HITL)
- ⇒ Guardrails/policy enforcement
- ⇒ Edge case Escalation

Difference b/w workflows and Agentic workflows

⇒ In workflows, LLMs are involved but we developers define the logic and sequence of tools which remains static for every time. LLMs are just used for content generation.

⇒ In Agentic workflows, we don't define the logic and sequence. LLM decides which tool to call and when and it is dynamic.

⇒ LangChain vs LangGraph

Challenges with LangChain

Challenge 01, Control Flow Complexity:

Difficult to make non-linear workflows as it don't support

- 1) Conditional Branches
- 2) Loops (Retrying)
- 3) Jumps

And for handling that we have to write custom python code called glue code which make things complex.

- Whereas, in **LangGraph** we have graph with nodes and edges and we can manage non-linear workflows.
- **Langgraph** also provide conditional edges and Loop Branching.

Challenge 02: Handling State

- ⇒ In workflows we have to maintain state in key value pairs like important datapoints in workflows
- ⇒ Langchain don't provide any mechanism for storing and handling state in key value pairs we have to do it manually using dictionary.
- ⇒ Where Lang Graph has state object for maintaining state and every node has full access of state object that's why state can be easily accessed, updated and handling within the whole workflow.

⇒ So Langchain is stateless,
where LangGraph is
stateful.

Challenge 03: Event Driven Execution

- In complex workflow we need Event Driven Execution
- But LangChain doesn't support event driven workflow it supports sequential workflow
- Where LangGraph provides event driven execution.

Challenge #04:

Fault Tolerance

- Fault Tolerance means if any failure occur we can handle that and resume from where the failure occur

Note from user

- LangChain doesn't provide pauser to handle
- But LangGraph supports fault tolerance

For small failures:

Retry Logic

For Large Failures:

Recovery as it uses checkpoints

Challenge 08:

HITL (Human in the Loop)

- As LangChain doesn't provide pauser (it is for short running workflows, not for long running workflows) so it doesn't support Human in the Loop.
- LangGraph supports Human in the Loop as it supports pauser

longtime because it supports stateful execution, checkpoints and resume logic.

Challenge 06 Nested Workflows

LangGraph supports Agentic Workflows as it supports subgraphs which becomes a node for another graph.

- So it helps us build multi-agent workflows.
- And because of subgraphs we can implement reusability.

Challenge 07 Observability

observability refers to how easily you can monitor, debug and understand your workflow what it is doing at runtime

- LangSmith is used for this feature.
- LangSmith is supported by Langchain but only langchain

things recorded not glue (custom) code so issue comes in large workflow due to which there is partial observability.

- Langgraph supports Langsmith property as ~~there~~ is no need of custom code so offers full observability.

Conclusion:

1. What is LangGraph?

LangGraph is an orchestration framework that enables you to build stateful, multi-step and event-driven workflows using LLMs. It is ideal for designing both single agent and multi-agent Agentic AI application.

⇒ Think of Langgraph as a flowchart engine for LLMs, you define the steps (nodes), how they are connected (edges) and the logic that governs the transition.

⇒ LangGraph takes care of state management, looping, conditional branching, pausing/resuming and fault recovery — features essential for building robust production-grade AI systems.

2. When to use what?

- Use langchain when you are building simple linear workflows like a prompt chain, summarizer or basic retrieval system.

- Use LangGraph when your case involves complex non-linear workflows that need:

- Conditional Paths
- Loops
- Human in the Loop steps
- Multi-agent coordination
- Event Driven Execution

3. Should we still use LangChain?

Yes, LangGraph is built on top of langchain, it doesn't replace it. You will still use components like

- Models
- Prompt Template
- Retrievers
- Document Loaders
- Tools etc.

LangGraph handles workflow orchestration where LangChain provides the building blocks for each step in workflow.

What is LangGraph - OneNote

FileHomeInsertDrawHistoryReviewViewHelp

What is LangGraph

LLM Workflows

Graphs, Nodes and Edges

State

Reducers

LangGraph Execution Model

What is LangGraph

07 July 202510:58

- LangGraph is an orchestration framework for building intelligent, stateful, and multi-step LLM workflows.
- It enables advanced features like parallelism, loops, branching, memory, and resumability — making it ideal for agentic and production-grade AI applications.
- It models your logic as a graph of nodes (tasks) and edges (routing) instead of a linear chain.

LLM workflow → graph

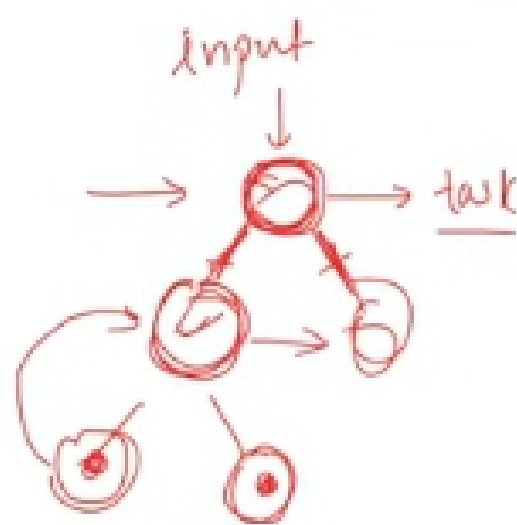
↓

flowchart execution

Search Notebooks

input

task



LLM Workflows - OneNote

FileHomeInsertDrawHistoryReviewViewHelp

Format Background

Full Page View

Buy Microsoft 365Sticky NotesShare

Add Page

What is LangGraph

LLM Workflows

Graphs, Nodes and Edges

State

Reducers

LangGraph Execution Model

behaviours like retries, multi-agent communication, or tool-augmented reasoning.

4. Common workflows

Prompt Chaining

In

LLM Call 1

Output 1

Gate

Pass

LLM Call 2

Output 2

LLM Call 3

Out

Fail

Exit

check

Routing

In

LLM Call Router

LLM Call 1

LLM Call 2

LLM Call 3

Out

topic → outline

topic → report

5000 words

report

LLMs

LLM Workflows - OneNote

FileHomeInsertDrawHistoryReviewViewHelp

Format Background

Add Page

What is LangGraph

LLM Workflows

Graphs, Nodes and Edges

State

Reducers

LangGraph Execution Model

Search Notebooks

LLM Workflows

07 July 2025 19:58

1. LLM workflows are a step by step process using which we can build complex LLM applications.
2. Each step in a workflow performs a distinct task — such as prompting, reasoning, tool calling, memory access, or decision-making.
3. Workflows can be linear, parallel, branched, or looped, allowing for complex behaviours like retries, multi-agent communication, or tool-augmented reasoning.
4. Common workflows

Prompt Chaining

workflow

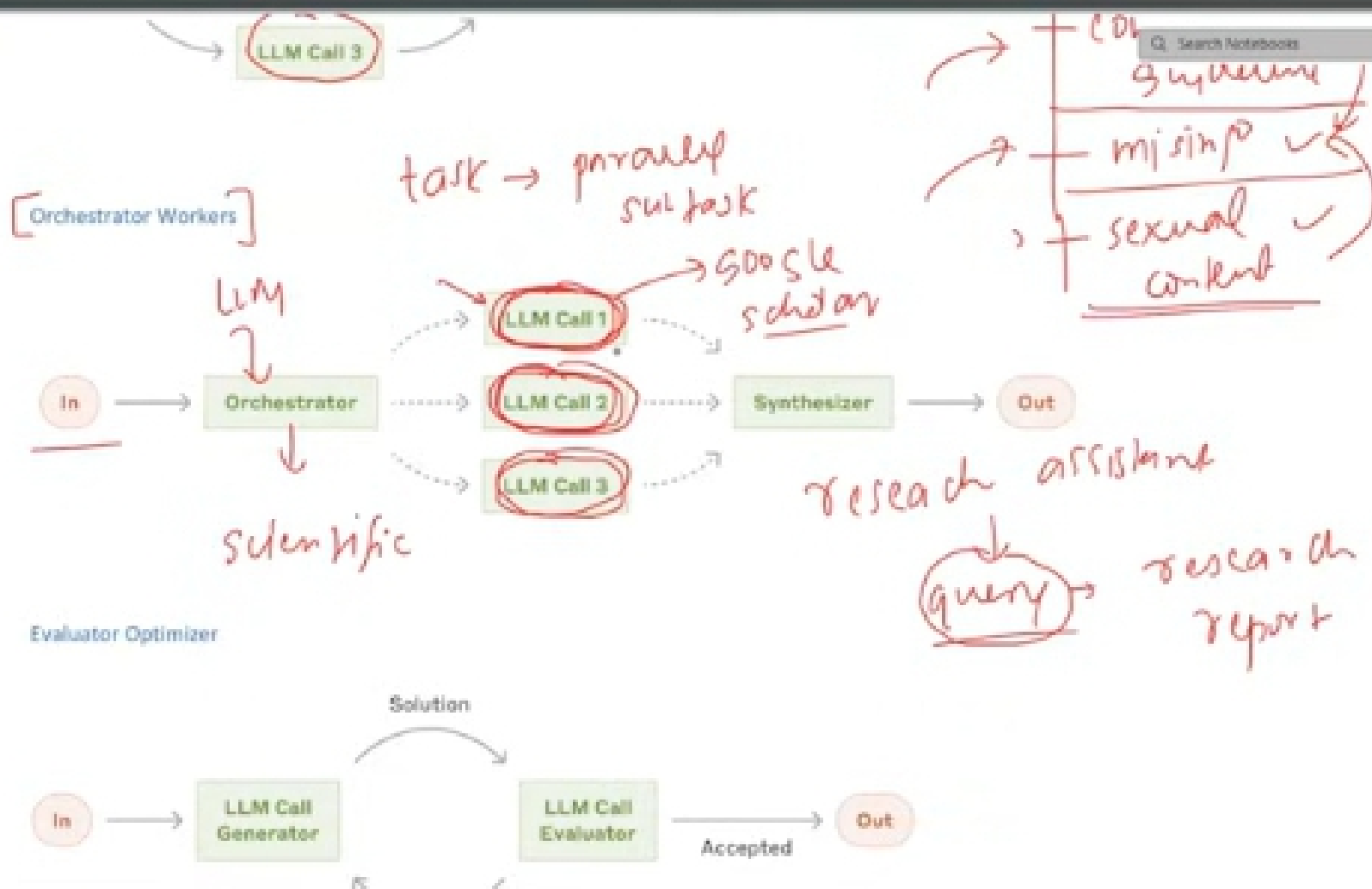
series of task
↓
goal

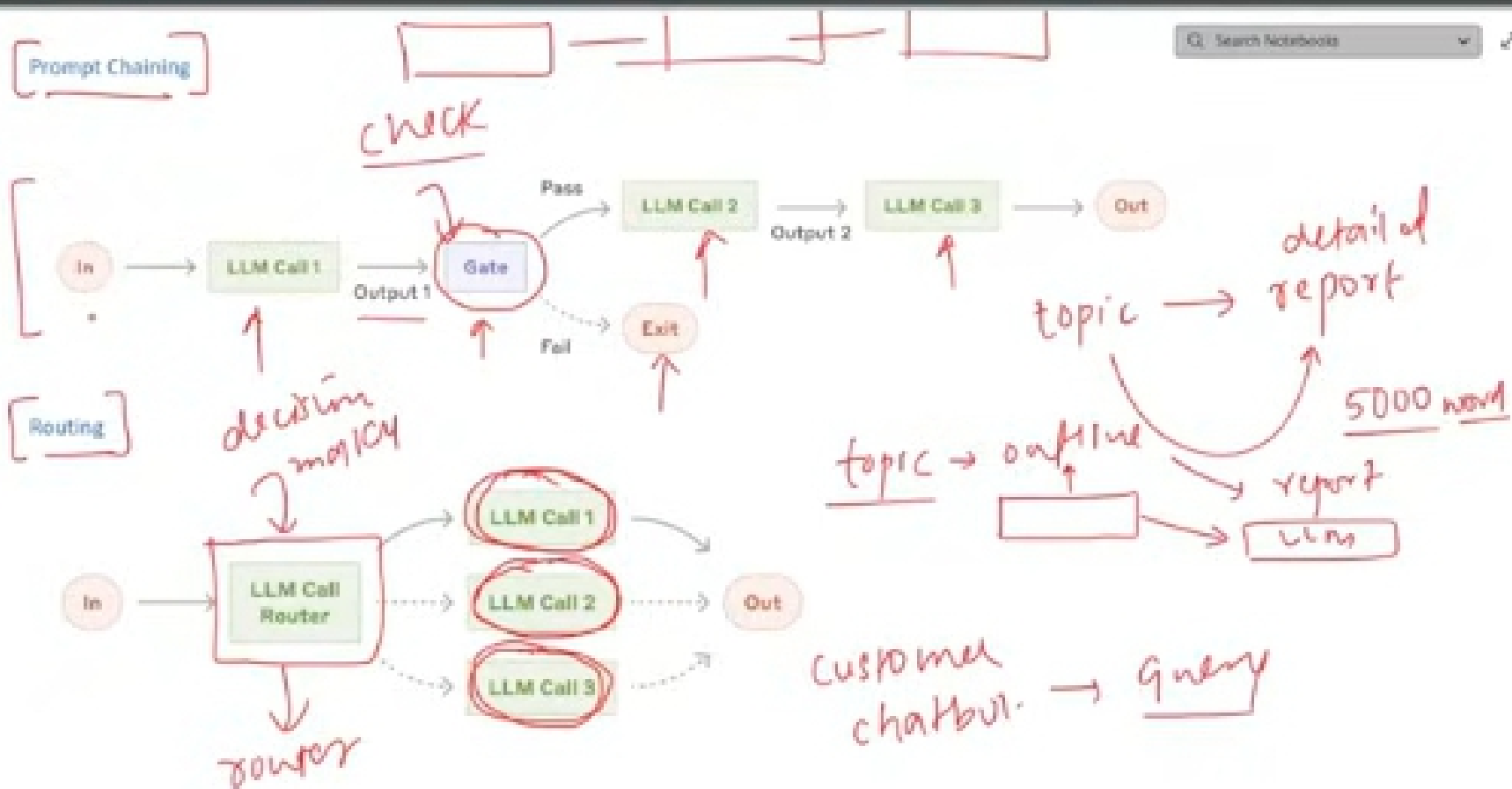
4

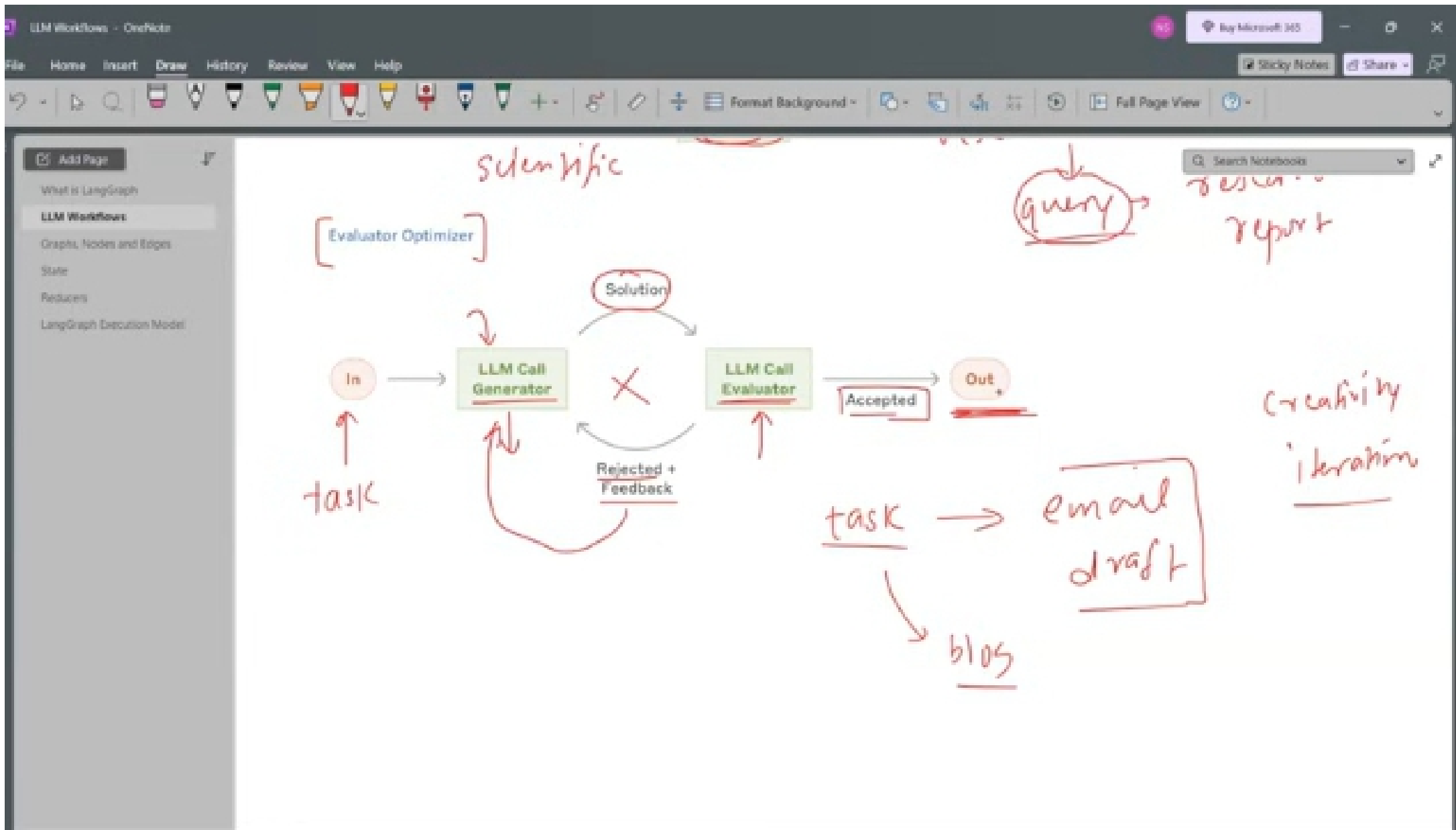
U.S. Department of Health and Human Services

100

LangGraph Execution Model







Graphs, Nodes and Edges - OneNote

FileHomeInsertDrawHistoryReviewViewHelp

Format Background

Full Page View

Buy Microsoft 365Sticky NotesShare

Add Page

What is LangGraph

LLM Workflows

Graphs, Nodes and Edges

State

Reducers

LangGraph Execution Model

Graphs, Nodes and Edges

07 July 202510:21

The system generates an essay topic, collects the student's submission, and evaluates it in parallel on depth of analysis, language quality, and clarity of thought. Based on the combined score, it either gives feedback for improvement or approves the essay.

1. **GenerateTopic**

→ System generates a relevant UPSC-style essay topic and presents it to the student.

2. **CollectEssay**

→ Student writes and submits the essay based on the generated topic.

3. **EvaluateEssay (Parallel Evaluation Block)**

→ Three evaluation tasks run in parallel.

• **EvaluateDepth** – Analyzes depth of analysis, argument strength, and critical thinking.

• **EvaluateLanguage** – Checks grammar, vocabulary, fluency, and tone.

• **EvaluateClarity** – Assesses coherence, logical flow, and clarity of thought.

4. **AggregateResults**

→ Combines the three scores and generates a total score (e.g., out of 15).

5. **ConditionalRouting**

→ Based on the total score:

• If score meets threshold → go to **ShowSuccess**

• If score is below threshold → go to **GiveFeedback**

6. **GiveFeedback**

→ Provides targeted suggestions for improvement in weak areas.

7. **CollectRevision (optional loop)**

→ Student resubmits the revised essay.

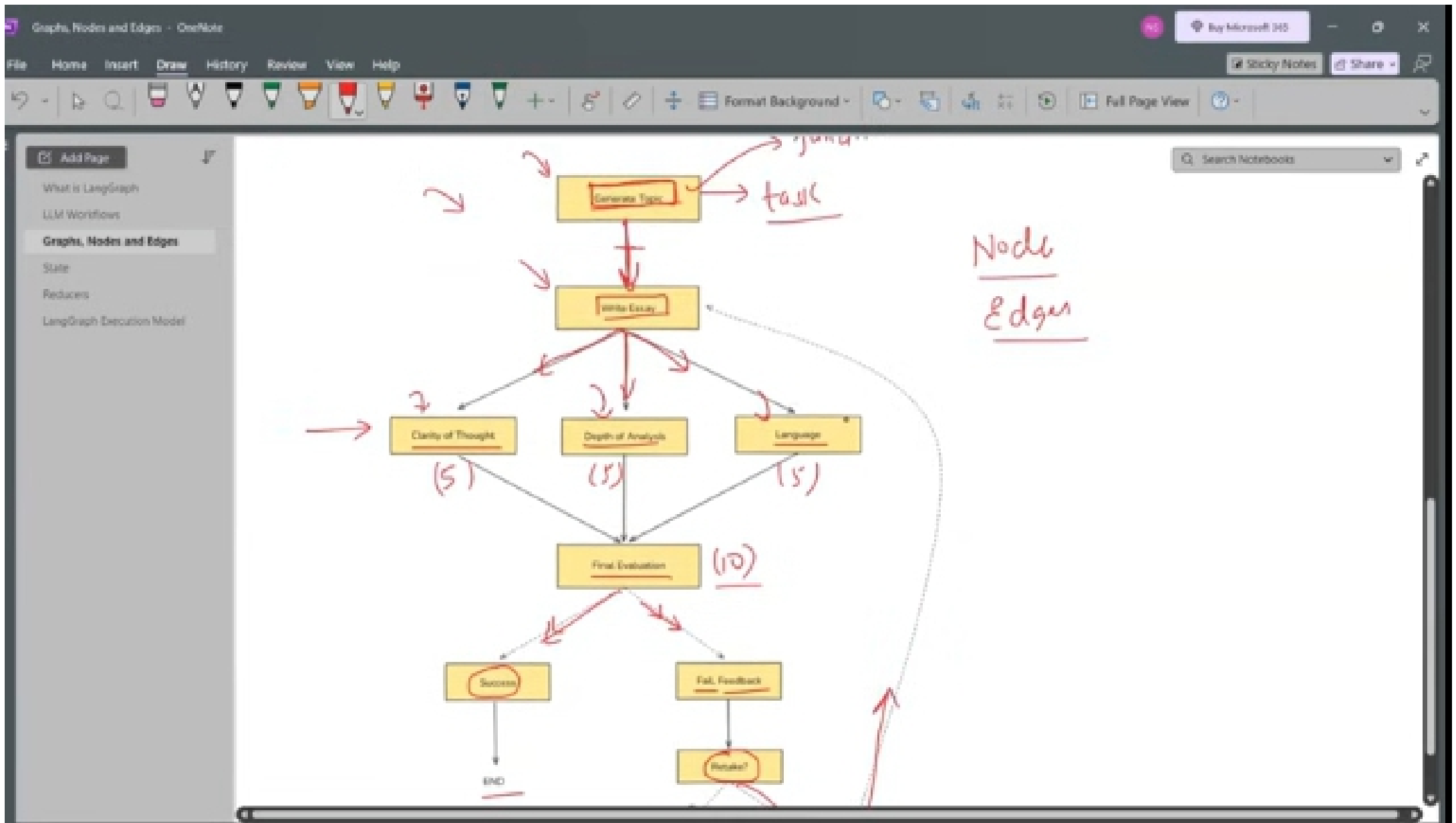
→ Loop back to **EvaluateEssay**

8. **ShowSuccess**

→ Congratulates the student and ends the flow.

UPSC → essays

(website) → practice



Add Page

What is LangGraph

LLM Workflows

Graphs, Nodes and Edges

State

Reducers

LangGraph Execution Model

State

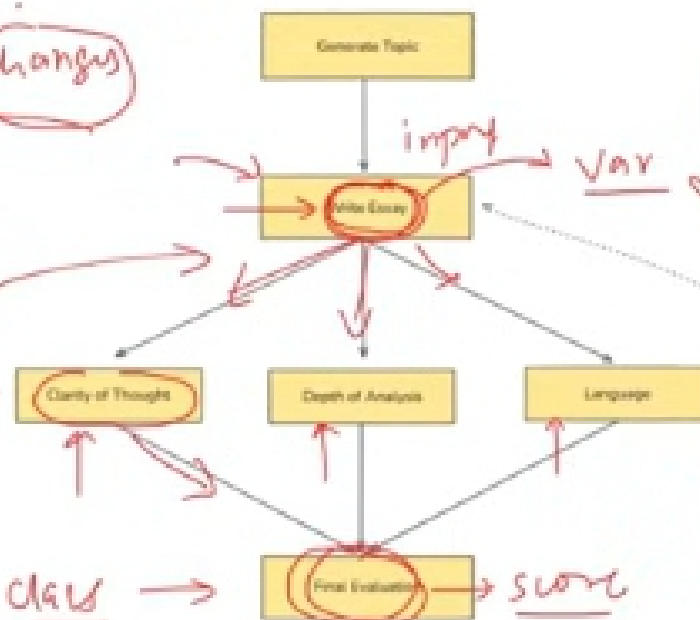
07 July 2025 19:54

In LangGraph, state is the shared memory that flows through your workflow — it holds all the data being passed between nodes as your graph runs.

```
essay_text: str
topic: str
depth_score: int
language_score: int
clarity_score: int
total_score: int
feedback: Annotated[list[str], add]
evaluation_round: int
```

evolvecode

dict → TypedDict class →

changes

graph → state
↓
powerful

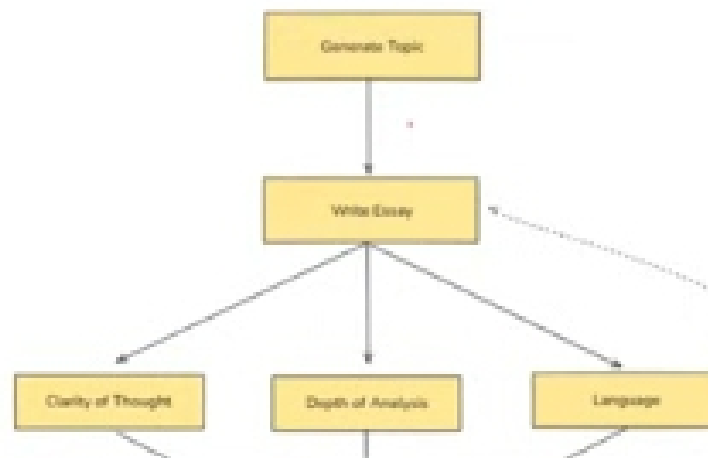
state
↓
shared
mutable

088 Page 20 of 20

Reducers in LangGraph define how updates from nodes are applied to the shared state.

Each key in the state can have its own reducer, which determines whether new data replaces, merges, or adds to the existing value.

```
essay_text: str
topic: str
depth_score: int
language_score: int
clarity_score: int
total_score: int
feedback: Annotated[list[str], add]
evaluation_round: int
```



LangGraph Execution Model - OneNote

FileHomeInsertDrawHistoryReviewViewHelp

↶

↷

🔍

🖌

🖌

🖌

🖌

🖌

🖌

🖌

🖌

🖌

+

✍

✂

+

Format Background

🔄

🔄

🔄

🔄

🔄

Full Page View

🔄

📄 Add Page

What is LangGraph

LLM Workflows

Graphs, Nodes and Edges

State

Reducers

LangGraph Execution Model

LangGraph Execution Model

08 July 2023 10:22

1. Graph Definition

You define:

- The state schema
- Nodes (functions that perform tasks)
- Edges (which node connects to which)

2. Compilation

You call `compile()` on the `StateGraph`.

This checks the graph structure and prepares it for execution.

3. Invocation

You run the graph with `invoke(initial_state)`.

LangGraph sends the initial state as a **message** to the entry node(s).

4. Super-Steps Begin

Execution proceeds in **rounds**.

Google Project → superstep

↓
system

large scale graph processing

state partially update

superstep

initial_state

↓

Generate Prompt

↓
edge

Write Essay

↓

Clarity of Thought

↓

Depth of Analysis

↓

Language

↓

Final Evaluation

↓

Success

Fail, Feedback

activate →

function

state partially update

graph structure

graph

LangGraph Execution Model - OneNote

FileHomeInsertDrawHistoryReviewViewHelp

↶

↷

🔍

🎨

🖌

🖌

🖌

🖌

🖌

🖌

🖌

+

✍

✂

+

Format Background

🔄

🔄

🔄

🔄

🔄

Full Page View

🔄

Add Page

What is LangGraph

LLM Workflows

Graphs, Nodes and Edges

State

Reducers

LangGraph Execution Model

4. Super-Steps Begin

Execution proceeds in **rounds**.
In each round (super-step):

- All **active nodes** (those that received messages) run in **parallel**
- Each returns an **update** (message) to the state

5. Message Passing & Node Activation

The messages are passed to downstream nodes via edges.
Nodes that receive messages become active for the next round.

6. Halting Condition

Execution stops when:

- No nodes are active, and
- No messages are in transit.

Search Notebooks

Success

End

Fail, Feedback

Retain?

No

Yes

End

Add Page

What is Persistence

How to implement Pers...

Code

Benefits of Persistence

What is Persistence

28 July 2023

fault tolerance

Persistence in LangGraph refers to the ability to save and restore the state of a workflow over time.

Database

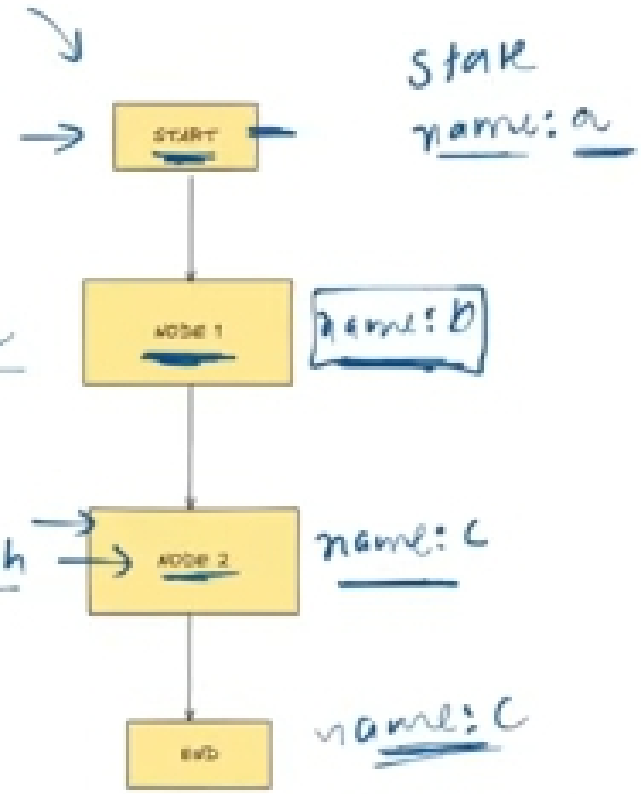
State
final-values
intermediate

name: c

crash

crash

persistence → save
↓
state



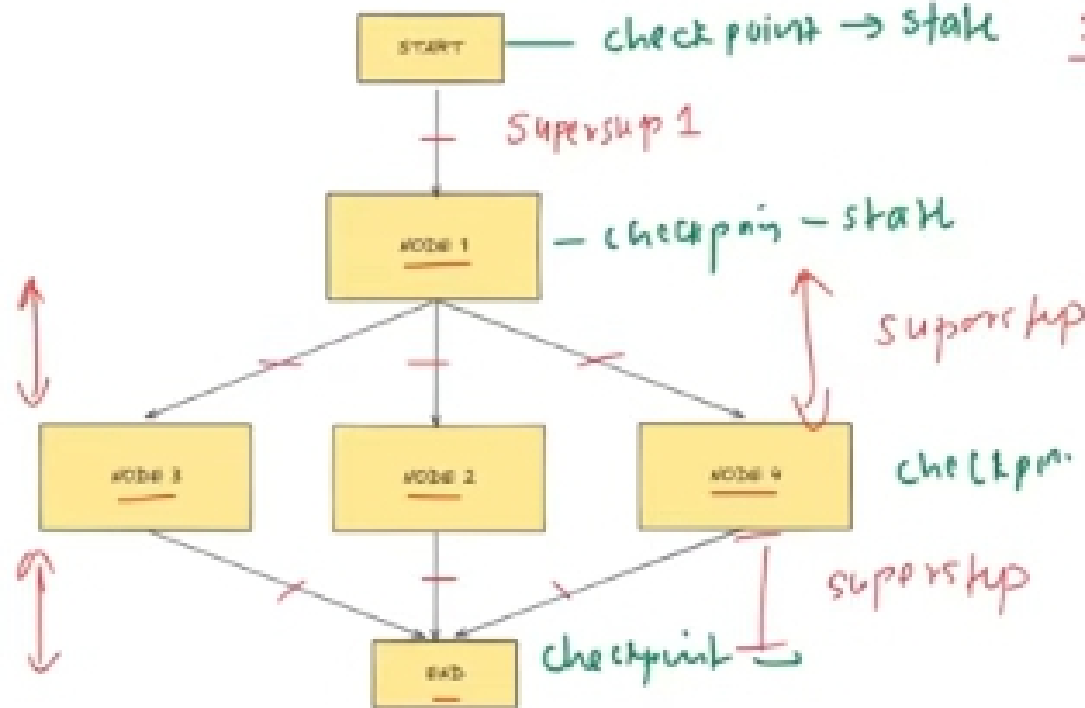
Checkpoints in Persistence

28 July 2024 13:17



checkpoints → state → save

superstep → checkpoint



save

conversational response

Observability is the ability to understand a system's internal state by examining its external outputs, like logs, metrics, and traces.

It allows you to diagnose issues, understand performance, and improve reliability by analyzing data generated by the system.

Essentially, it's about being able to answer "why" something is happening within a system, even if you didn't anticipate the problem.

AIM → non-deterministic

input → output

input → diff output

software → execute

trace → store →

What does LangSmith Trace?

Friday, 7th August 2025 10:11 AM

1. Input and Output ✓
2. All the intermediate steps ✓
3. Latency ✓
4. Token usage ✓
5. Cost ✓
6. Error ✓
7. Tags ✓ *
8. Metadata ✓
9. Feedback ✓

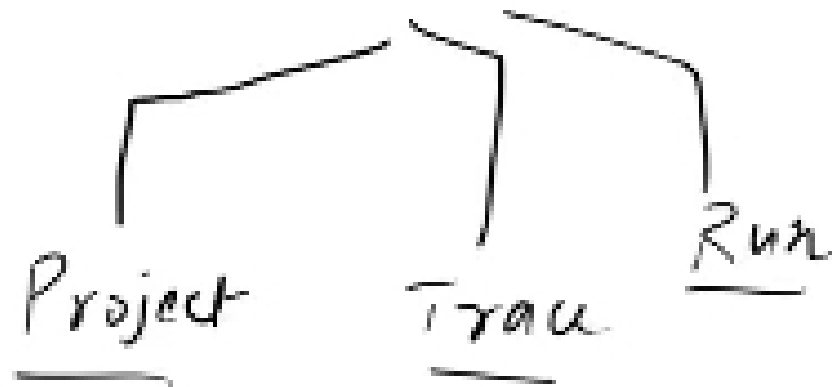
What is LangSmith

Friday, 15 August 2025 8:11 AM

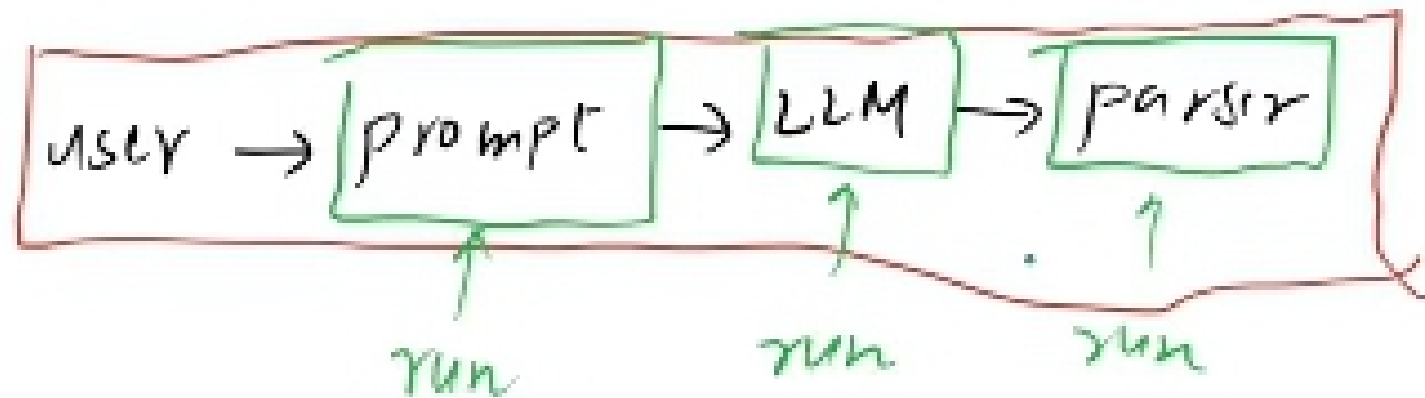
LangSmith is a unified observability & evaluation platform where teams can debug, test, and monitor AI app performance.

Core Concepts

Friday, 15 August 2025 8:27 AM



execute
↓
trace



Code Examples

Friday, 15 August 2025 8:11 AM

RAG apps have two big failure modes:

1. Retriever errors – wrong / irrelevant docs retrieved.
2. Generator errors – model hallucinates or misuses context.

In production, it's often unclear where the failure happened. Was the retriever bad, or did the LLM ignore the docs?

• LangSmith automatically records:

- User query
- Retrieved documents
- LLM prompt (with inserted docs)
- LLM response

question
└───> rel docs

question

prompt → LLM →

Other Features of LangSmith

Friday, 25 August 2023

8:12 AM

Monitoring and Alerting

What it does:

Monitoring in LangSmith looks across many traces at once to track the overall health of your LLM system. It aggregates key metrics like latency (P50, P95, P99), token usage, cost, error rates, and success rates. You can set up alerts to notify you when these metrics drift outside acceptable ranges (e.g., a spike in latency, higher error rates, or unexpected cost growth).

Why it matters:

In production, issues often appear first as patterns across multiple runs rather than in a single trace. Monitoring helps you catch these early signals before they impact users at scale. Instead of waiting for customer complaints, you're proactively alerted when performance degrades or costs spike, enabling faster response and more reliable applications.

57

multiple traces

monitor

observ



LLM → exe



trace →

Prompt Experimentation

What it does:

Prompt Experimentation in LangSmith allows you to systematically test and compare different prompt versions. You can run A/B tests across prompts on the same dataset, track their performance against evaluation metrics, and record the outcomes. Results are stored over time, giving you a clear history of which prompt variations worked best and under what conditions.

Evaluation

What it does:

Evaluation in LangSmith helps you systematically measure the quality of your LLM outputs. You can run tests against gold-standard datasets or apply custom evaluation metrics such as faithfulness, relevance, or completeness. LangSmith supports multiple approaches: automated scoring with LLM-as-a-judge, semantic similarity checks, or even custom Python evaluators. (Evaluations can be run both offline (batch tests before deployment) and online (continuous checks on live traffic)).

Why it matters:

LLM behavior can be unpredictable — a small change in prompts, models, or retrieval logic may improve some cases but break others. Evaluation provides an objective, repeatable way to track performance over time, ensuring that new versions are actually better and preventing regressions.

Example:

For a RAG chatbot, you might evaluate:

- **Faithfulness** → Are answers grounded in retrieved documents?
- **Relevance** → Did the response actually address the user's question?

By running the same dataset across GPT-4, Claude, and LLaMA, you can directly compare which model (or pipeline setup) performs best.

Dataset Creation & Annotation

What it does:

- Provides tools to build datasets for evaluation and fine-tuning.
- Supports manual annotation (e.g., labeling whether an answer is correct).
- Stores datasets versioned for reuse across projects.

Why it matters:

High-quality datasets are critical for evaluation and feedback loops.

Example:

- Customer support: Build a dataset of **common questions + expected answers**.
- Use it to benchmark your RAG agent every time you change retrieval logic.

Example:

- Customer support: Build a dataset of common questions + expected answers.
- Use it to benchmark your RAG agent every time you change retrieval logic.

[User Feedback Integration]

What it does:

- Lets you capture thumbs up/down, ratings, or structured feedback from users in production.
- Feedback is logged alongside traces → tied to the exact prompt, model, and state.
- Supports bulk analysis of what users like/dislike.

Collaboration

What it does:

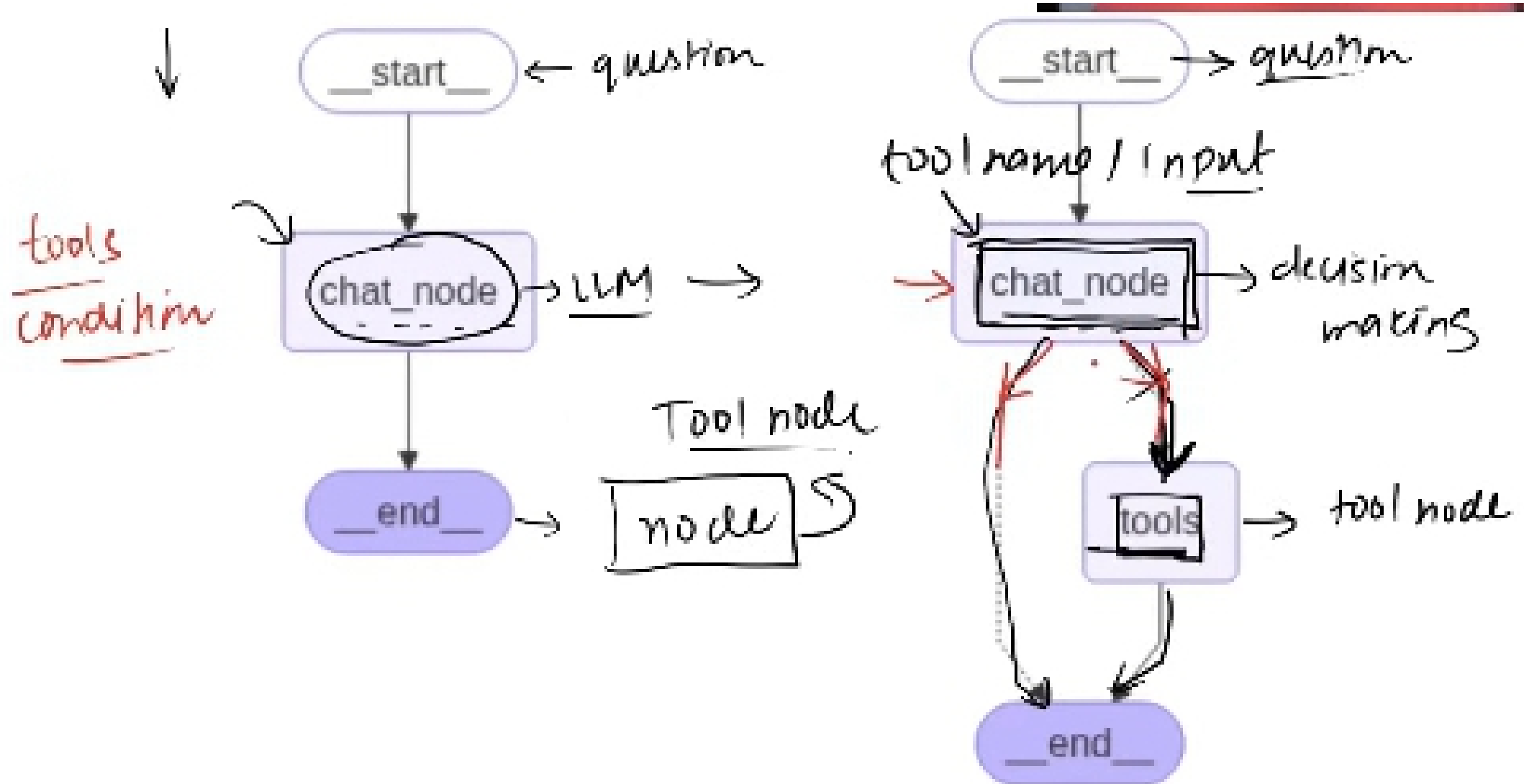
- Team members can view, share, and comment on traces, datasets, and evaluations.
 - Provides a web UI where non-engineers (PMs, QA, annotators) can inspect and annotate runs.
 - Enables shared experiment dashboards.
-

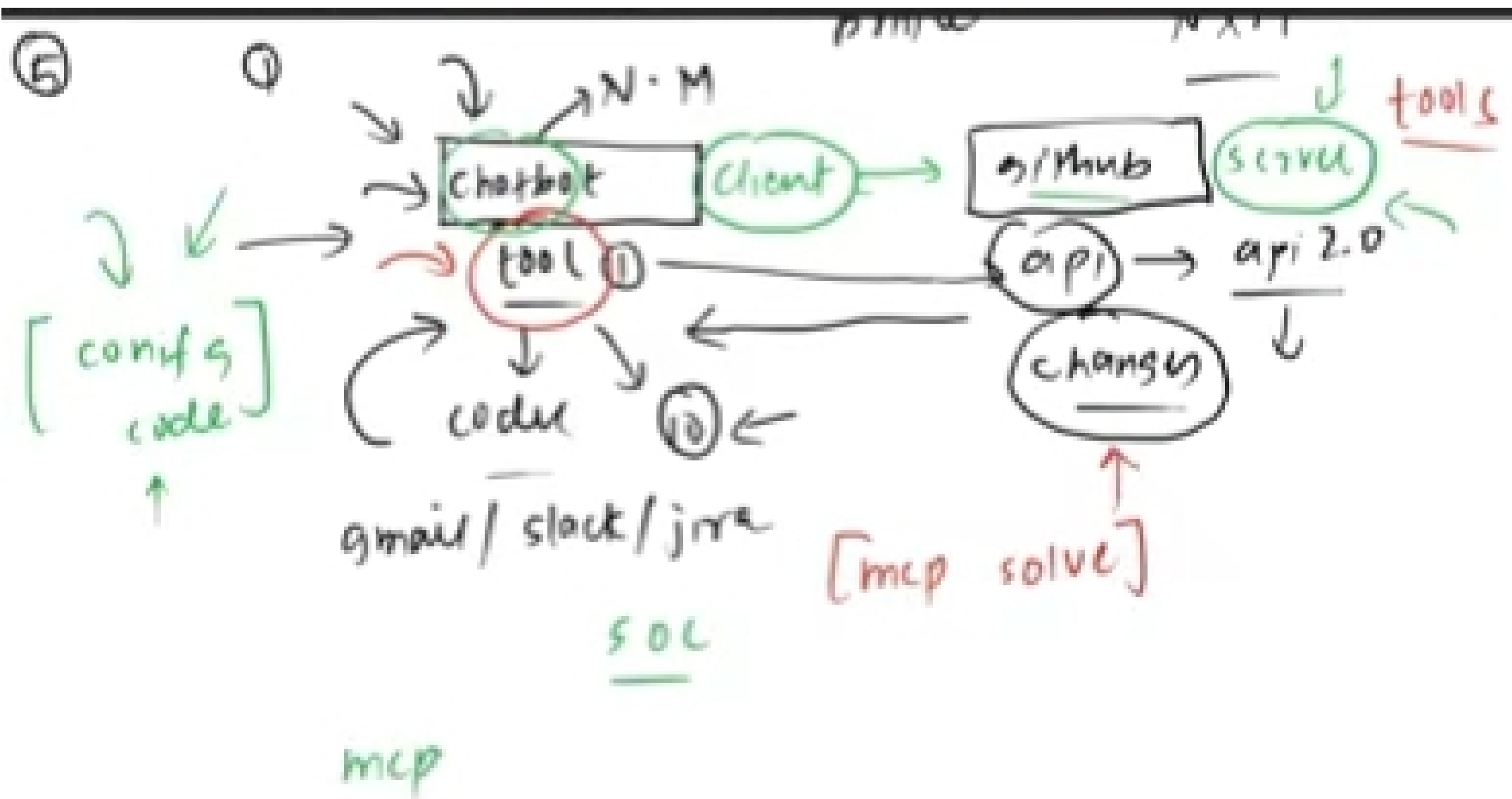
In LangGraph, a ToolNode is a prebuilt node type that acts as a bridge between your graph and external tools (functions, APIs, utilities).

- Normally in LangGraph you'd write a node function yourself: it takes in state and returns state.
- A ToolNode is a ready-made node that knows how to handle a list of LangChain tools.
- Its job: listen for tool calls from the LLM (like "call search()" or "get_weather()") and automatically route the request to the correct tool, then pass the tool's output back into the graph.

tools_condition is a prebuilt conditional edge function that helps your graph decide:

"Should the flow go to the ToolNode next, or back to the LLM?"





OneNote

Home Insert Draw View Tell me

Text Mode Lasso Select Insert Eraser Pen Marker Highlighter Ink Colour 0.25 mm 0.50 mm 0.5 mm 0.7 mm 1 mm

What is HITL

Tuesday, 9 December 2025 10:57 PM

HITL (Human-in-the-Loop) is a design approach in AI systems where a human actively participates at critical points of the AI workflow — either to **supervise, approve, correct, or guide** the model's output.

Think of HITL as putting a human "checkpoint" inside an AI pipeline so that important decisions are not made autonomously by the model.

Why HITL Exists?

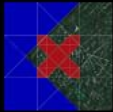
1. To help agentic systems
2. To add accountability

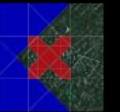
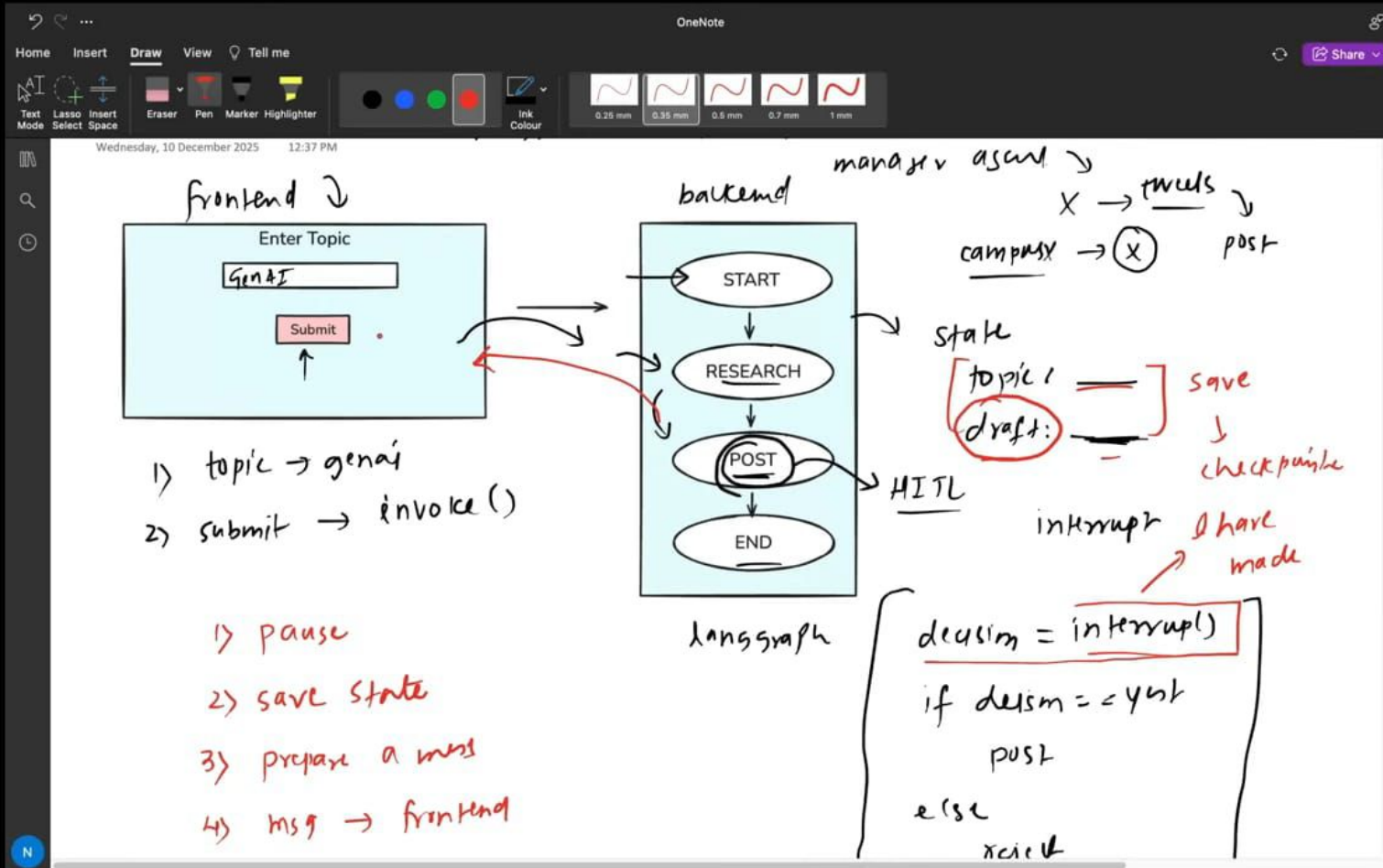
HITL ensures:

- Accuracy
- Safety
- Ethical alignment
- Better user experience

Common HITL Patterns:

- Action Approval Pattern (Approve / Reject Before Execution)
- Output Review / Edit Pattern
- Ambiguity Clarification Pattern
- Escalation Pattern





OneNote

Home Insert Draw View Tell me

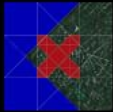
Text Mode Lasso Select Insert Space Eraser Pen Marker Highlighter Ink Colour 0.25 mm 0.35 mm 0.5 mm 0.7 mm 1 mm

3) prepare a msg
4) msg → frontend

1) receive the interrupt
2) user — input
3) invoke (command = yes/no)

post
else
reject

interrupt
command



OneNote

Home Insert Draw View Tell me

Text Mode Lasso Select Insert Space Eraser Pen Marker Highlighter Ink Colour 0.25 mm 0.35 mm 0.5 mm 0.7 mm 1 mm

What are Subgraphs?

Monday, 15 December 2025 11:28 PM

A subgraph in LangGraph usually means a graph that is embedded and executed as a node inside another graph.

Sub Graph 1

Sub Graph 2

Subgraphs

- concept
- Need
- practical

nodes (tasks)

subgraph

OneNote

Home Insert Draw View Tell me

Text Mode Lasso Select Insert Space Eraser Pen Marker Highlighter Ink Colour 0.25 mm 0.35 mm 0.5 mm 0.7 mm 1 mm

Why Subgraphs are needed?

User → LLM → Output

→ Tool calls —
→ RAG
→ Conditional routing
→ Retries —
→ Memory —
→ HITL —
→ Evaluation
→ Guardrails —

state

subgraph

[Soft dev agent]

TL → agent

BT → Test

FT → Test

code → code → prev → dev

codins

Test

1) modularity
2) reusability
3) Maintainability

1) Failure Isolation
2) State separation
3) Observability

Chrome File Edit View History Bookmarks Profiles Tab Window Help

YouTube Video Sc... Subgraphs - Docs Subgraph in Lang... ChatGPT - Reality (145) Quantum Inf... Subgraphs - Docs Introduction

docs.langchain.com/oss/python/langgraph/use-subgraphs

GenAI XGBoost MLOps Deep Learning OldMapsOnline All Bookmarks

Share how you're building agents for a chance to win LangChain swag!

LangChain Docs LangChain + LangGra... Search... Ask AI GitHub Try LangSmith

LangChain LangGraph Deep Agents Integrations Learn Reference Contribute

Time travel
Memory
Subgraphs
Production
Application structure
Test
LangSmith Studio
Agent Chat UI
LangSmith Deployment
LangSmith Observability

LangGraph APIs
Graph API
Choosing APIs
Graph API
Use the graph API
Functional API
Runtime

parts of the graph independently, you can define each part as a subgraph, and as long as the subgraph interface (the input and output schemas) is respected, the parent graph can be built without knowing any details of the subgraph

When adding subgraphs, you need to define how the parent graph and the subgraph communicate:

- **Invoke a graph from a node** — subgraphs are called from inside a node in the parent graph
- **Add a graph as a node** — a subgraph is added directly as a node in the parent and **shares state keys** with the parent

Setup

```
pip uv  
pip install -U langgraph
```

Set up LangSmith for LangGraph development Sign up for [LangSmith](#) to quickly spot issues and improve the performance of your LangGraph projects. LangSmith lets you use trace data to debug, test, and monitor your LLM apps built with LangGraph — read more about how to get started [here](#).

On this page
Setup
Invoke a graph from a node
Add a graph as a node
Add persistence
View subgraph state
Stream subgraph outputs

OneNote

Home Insert Draw View Tell me

Text Mode Lasso Select Insert Eraser Pen Marker Highlighter Ink Colour 0.35 mm 0.35 mm 0.5 mm 0.7 mm 1 mm

Types of Subgraph

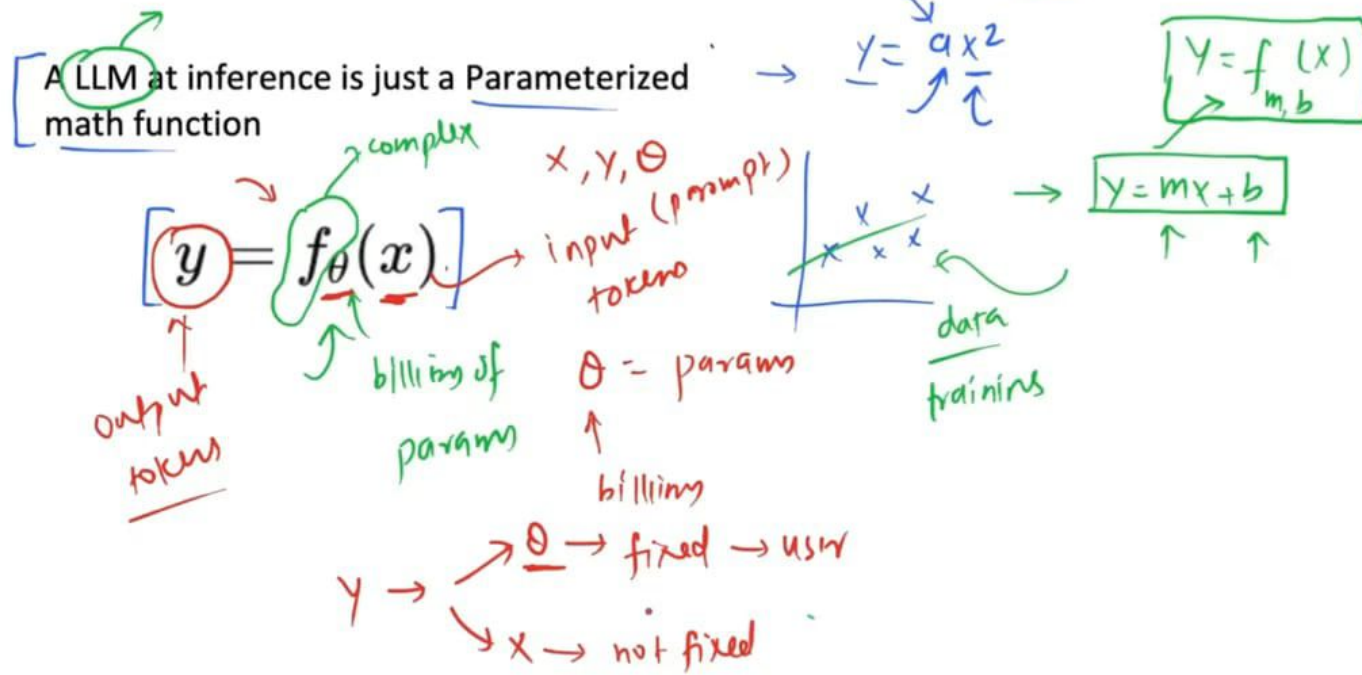
Monday, 15 December 2025 11:28 PM

The diagram illustrates two types of subgraphs:

- separate state:** A tree structure with red nodes. A blue circle highlights a node, with the word "invoke" written next to it. A blue arrow points from this node to a separate, smaller tree structure with blue nodes. A green checkmark is above the root of the first tree, and another green checkmark is above the root of the second tree.
- shared state:** A tree structure with red nodes. A blue box highlights a subtree. A green arrow points from the root of the tree to the blue box. The words "shared state" are written next to the box. A green arrow points from the bottom of the box back to the root of the tree.

LLM Memory

Monday, 22 December 2025 6:45 PM



$x \rightarrow$ not fixed

But this Parameterized math function is
Stateless

$$[y = f_{\theta}(x)]$$

$$y_1 = f_{\theta}(x_1)$$

$$y_2 = f_{\theta}(x_2)$$

A system is **stateless** if it's output depends only on the current input,
and not on anything that happened before.



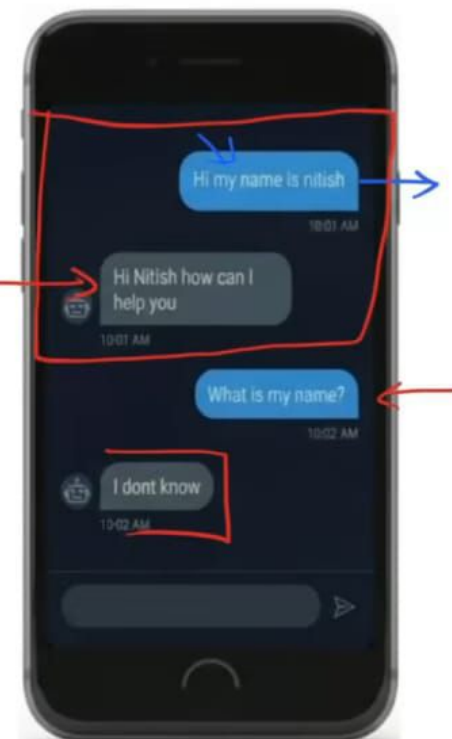
Fact - LLMs don't have any intrinsic Memory

Deadlock

fact 3 → externally

LLM → memory
↳ first

$$y = f_{\theta}(x)$$



The Solution



$$[y = f_{\theta}(x)]$$

$$y_1 = f_{\theta}(x_1)$$

$$y_2 = f_{\theta}(x_2)$$

$$y_2 = f_{\theta}(\text{con}(x_1, y_1, x_2))$$

[Context Window] - The context window is the amount of text an LLM can read and remember at one time before answering.

$[y = f_{\theta}(x)]$

a lot of tokens

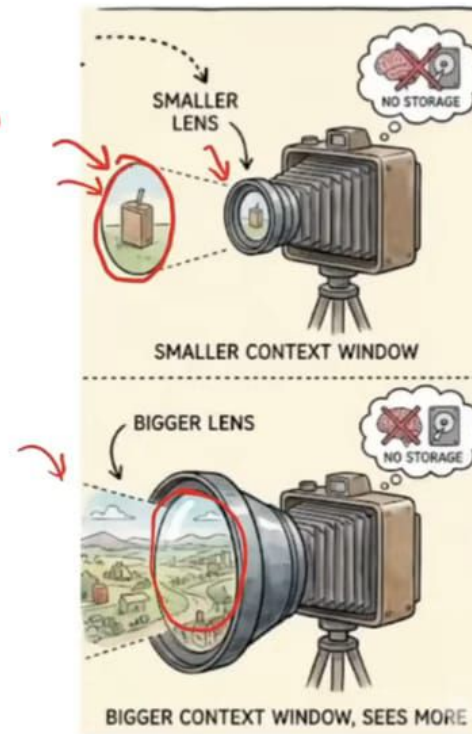
input tokens

max tokens

[128k tokens] IM →

200 pages pdf

camera
↓
LLM
context window



Text Mode
Lasso Select
Insert Space

Eraser Pen Marker Highlighter

Ink Colour

0.25 mm

0.35 mm

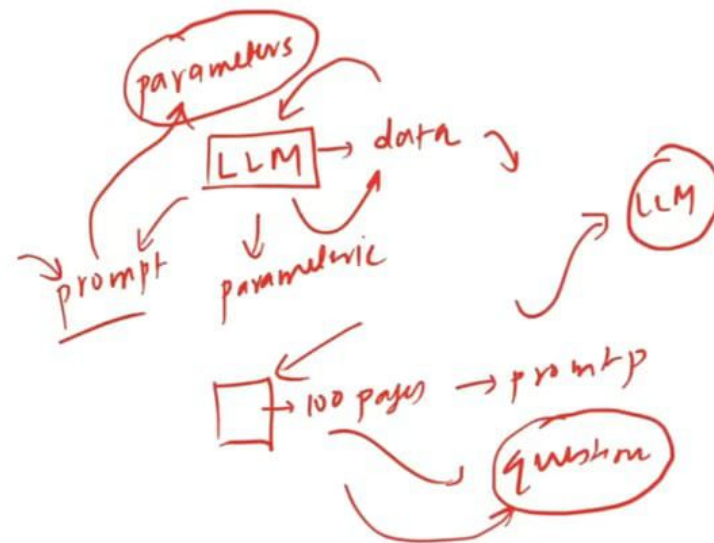
0.5 mm

0.7 mm

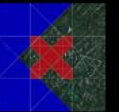
1 mm

BIGGER CONTEXT WINDOW, SEES MORE

In-context learning is an emergent ability that allows an LLM to use information and patterns present in the prompt itself, in addition to its trained parametric knowledge, to generate an answer.



N



Text Mode

Lasso Select Space

Eraser

Pen

Marker

Highlighter

Ink Colour

0.25 mm

0.35 mm

0.5 mm

0.7 mm

1 mm

STM

turning-point analysis cric...

WhatsApp message draft

Better name suggestions

Pip installs list

Thumbnail generation requ...

Egg and papaya combo

Kolkata tram history

Vertex AI explained

Top residential societies Gu...

Google AI History

Project Marvin inquiry

Nomination letter importance

Google Research vs Deep...

AI ecosystem summary

Add RAG to Chatbot

Nitish Singh

ChatGPT 5.2

OpenAI + Messages

(Installed automatically with langchain-openai, but listing separately for clarity)

bash

pip install openai

Optional (but recommended)

If you plan to use notebooks / debugging:

bash

pip install jupyter

STM [thread] scope

concal [message] empty

1000's → message → too long

conversation

Final List (Copy-Paste Ready)

bash

pip install langchain langchain-openai langchain-community langgraph python-dotenv faiss-

If you want, I can prepare a requirements.txt for this entire RAG + LangGraph chatbot setup.

+ Ask anything

Problems with Short Term Memory

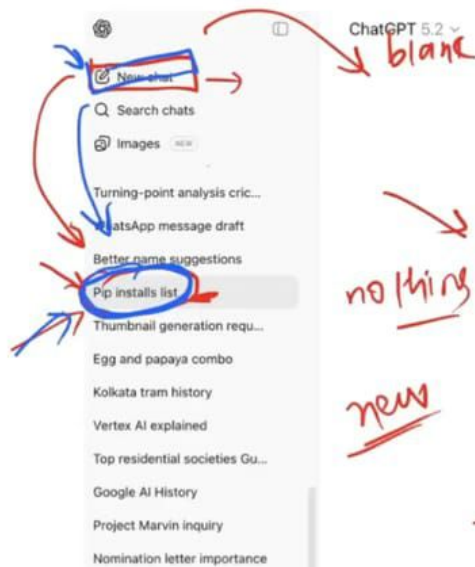
1. Short Term Memory is Fragile

STM → implements

message → code rent / server

possible

STM → database



OpenAI + Messages

(Installed automatically with langchain-openai, but listing separately for clarity)

```
bash
pip install openai
```

Optional (but recommended)

If you plan to use notebooks / debugging:

```
bash
pip install jupyter
```

Final List (Copy-Paste Ready)

```
bash
pip install langchain langchain-openai langchain-community langgraph python-dotenv faiss-
```

Thread id → 2

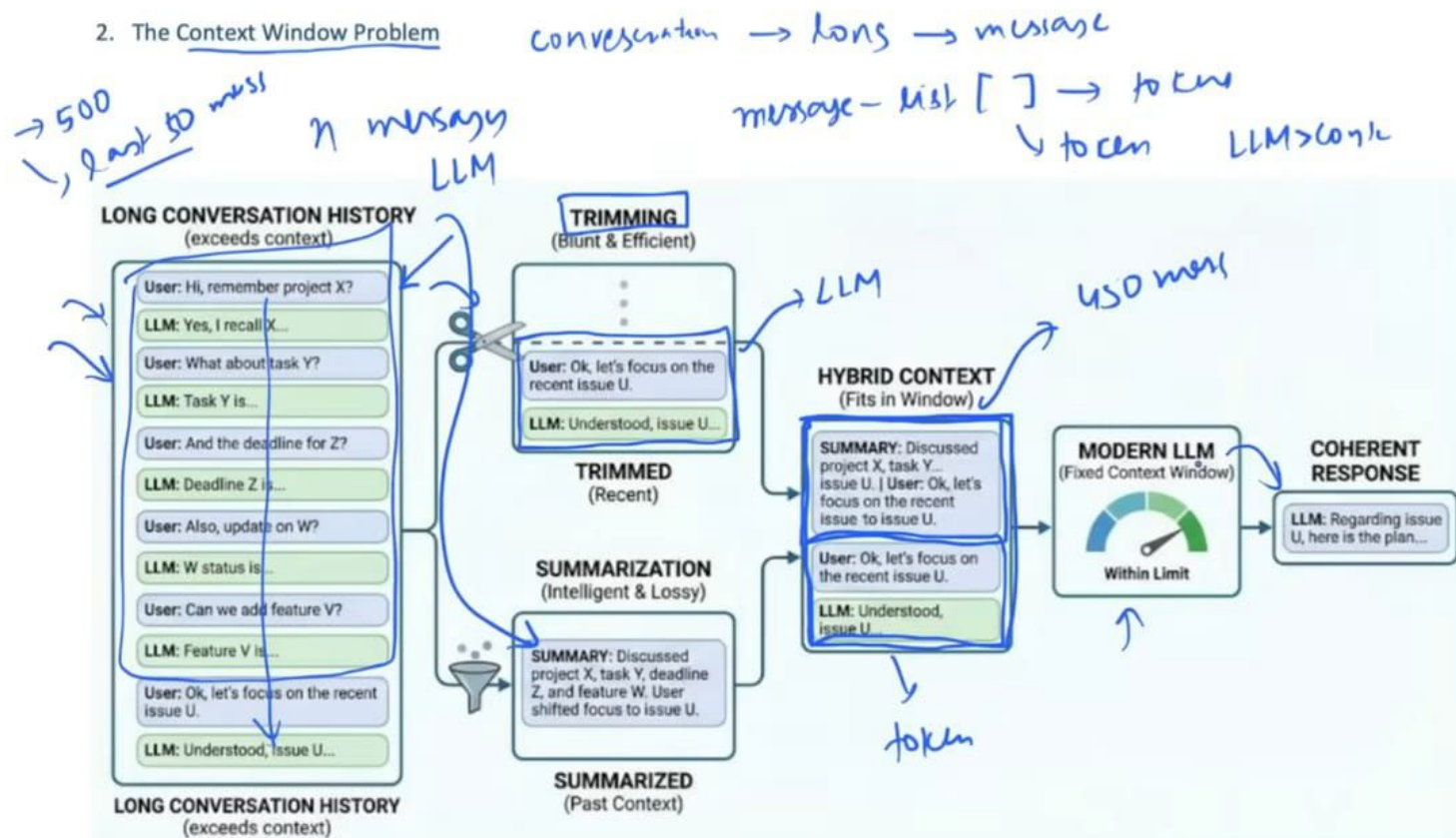
message

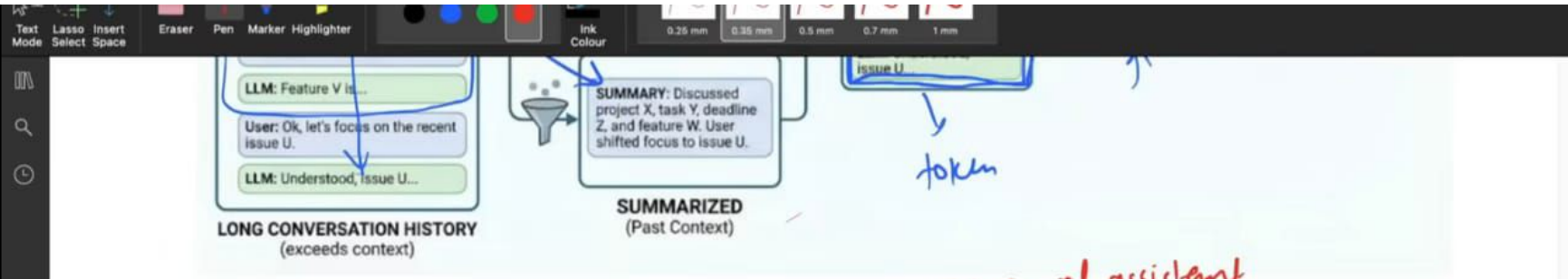
remove

Thread id → 1

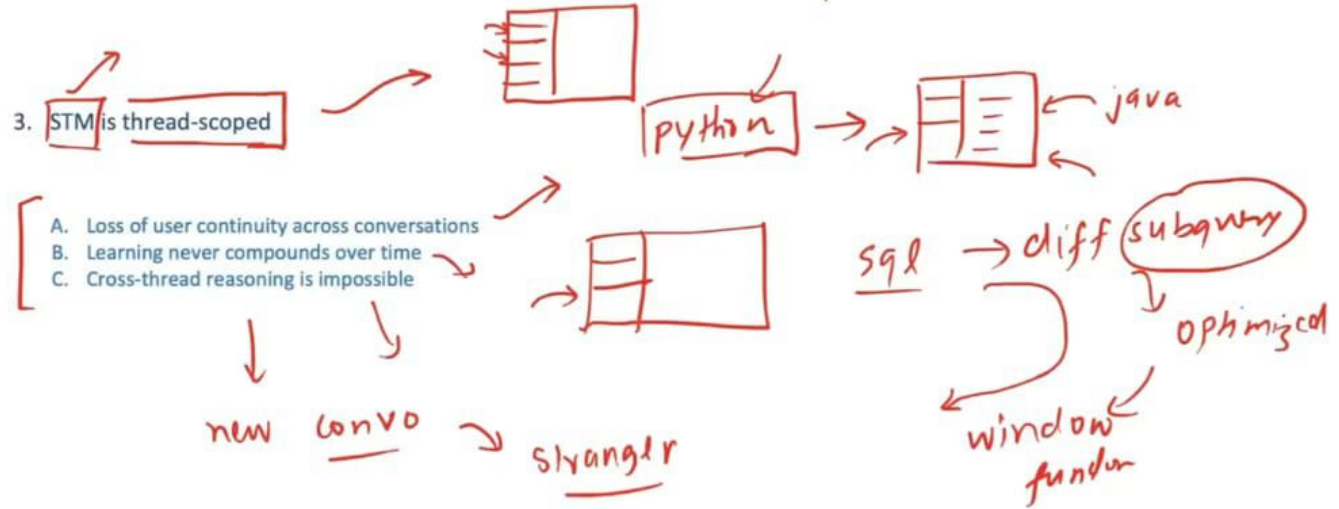


2. The Context Window Problem





personal assistant!



Solution - We need a new kind of memory

Information that should survive:

- beyond a single conversation
- beyond a single session
- sometimes even beyond days or weeks

This is the kind of information that defines *continuity*.

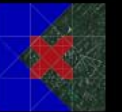
Things like:

- who the user is
- how the system is expected to behave for them
- what tends to work and what usually fails
- what decisions were already made in the past

It has to be selective.

Only information that proves to be stable, useful, and reusable should survive beyond a single thread. Everything else should naturally fade away.

Long Term Memory



Types of Long Term Memory

Three Types of Long-Term Memory (LTM) in LLM Systems



1 Episodic Memory — what happened

What it stores



Past events and experiences, usually tied to time or context.



Examples: "Yesterday, the user rejected solution A", "The last deployment failed due to missing credentials", "In the previous session, this approach worked", "User approved plan X in the last conversation"

Why it exists



Answers questions like: "What did we do last time?", "What happened before this?", "Have we tried this already?"



STM cannot answer these across threads. Episodic LTM can.



2 Semantic Memory — what is true

What it stores



Facts, knowledge, and stable information that are not tied to a single event.



Examples: "User prefers Python", "This user is a beginner", "The system uses PostgreSQL", "Budget constraint is ₹10k"

Why it exists



Answers questions like: "What do we know about this user?", "What constraints always apply?", "What facts should persist?"



This is the most common form of LTM in products.



3 Procedural Memory — how to do things

What it stores



Strategies, rules, and learned behaviors.



Examples: "When solving SQL problems for this user, avoid subqueries", "If tool X fails, retry with tool Y", "For this user, explain step by step", "Preferred workflow for task Z"

Why it exists



Answers questions like: "What approach usually works?", "How should I behave in this situation?", "What strategy should I follow?"

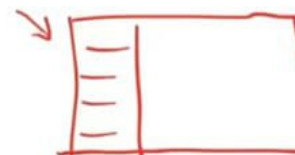


This is what enables learning that compounds over time.

How does Long Term Memory work?

Step 1 - Creation/update

1) creation
2) storage
3) get
4) Injection



"Is anything from what just happened worth remembering beyond this conversation?"

What the system looks at

- User messages ✓
- Model responses ✓
- Tool outcomes ✓

What happens in this step

The system:

- extracts memory candidates ✓
- filters out noise
- decides scope (user / agent / app)
- decides whether to:
 - create a new memory ✓
 - update an existing one ✓
 - ignore it ✗

Step 2 - Storage



Step 2 - Storage

→ store

"Where and how do we keep this memory?"

What storage actually means

- Writing memory to a durable store
- Assigning identifiers and metadata
- Making it survive restarts and crashes

Depending on the memory type, this might be:

- a relational database →
- a key-value store →
- a log →
- a vector database (for semantic lookup)

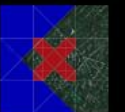
I

Step 3 - Retrieval

"Given the current situation, what should I remember right now?"

What retrieval involves

The system:



Text Mode

Lasso Select

Insert Space

Eraser

Pen

Marker

Highlighter

Ink Colour

0.25 mm

0.35 mm

0.5 mm

0.7 mm

1 mm

96

• Making it survive restarts and crashes

• a log →

• a vector database (for semantic lookup)

Step 3 - Retrieval

“Given the current situation, what should I remember right now?”

-	-	-
-	-	-
-	-	-

What retrieval involves

The system:

• looks at the current input

• decides whether memory is needed

• searches memory stores

• selects a small, relevant subset

Key point:

Retrieval is selective, not exhaustive.

Step 4 - Injection



Step 4 - Injection

"How does memory actually influence the model?"

→ What injection really is

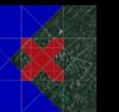
- Retrieved memory is inserted into STM
- It becomes part of the prompt
- The model sees it as just more tokens

The Challenges

1. Deciding what is worth remembering
2. Retrieving the right memory at the right time
3. Orchestrating the entire system

LTM → LLM

LTM → search → STM → context → LLM



OneNote

Home Insert Draw View Tell me

Text Mode Lasso Select Space Eraser Pen Marker Highlighter Ink Colour 0.25 mm 0.35 mm 0.5 mm 0.7 mm 1 mm

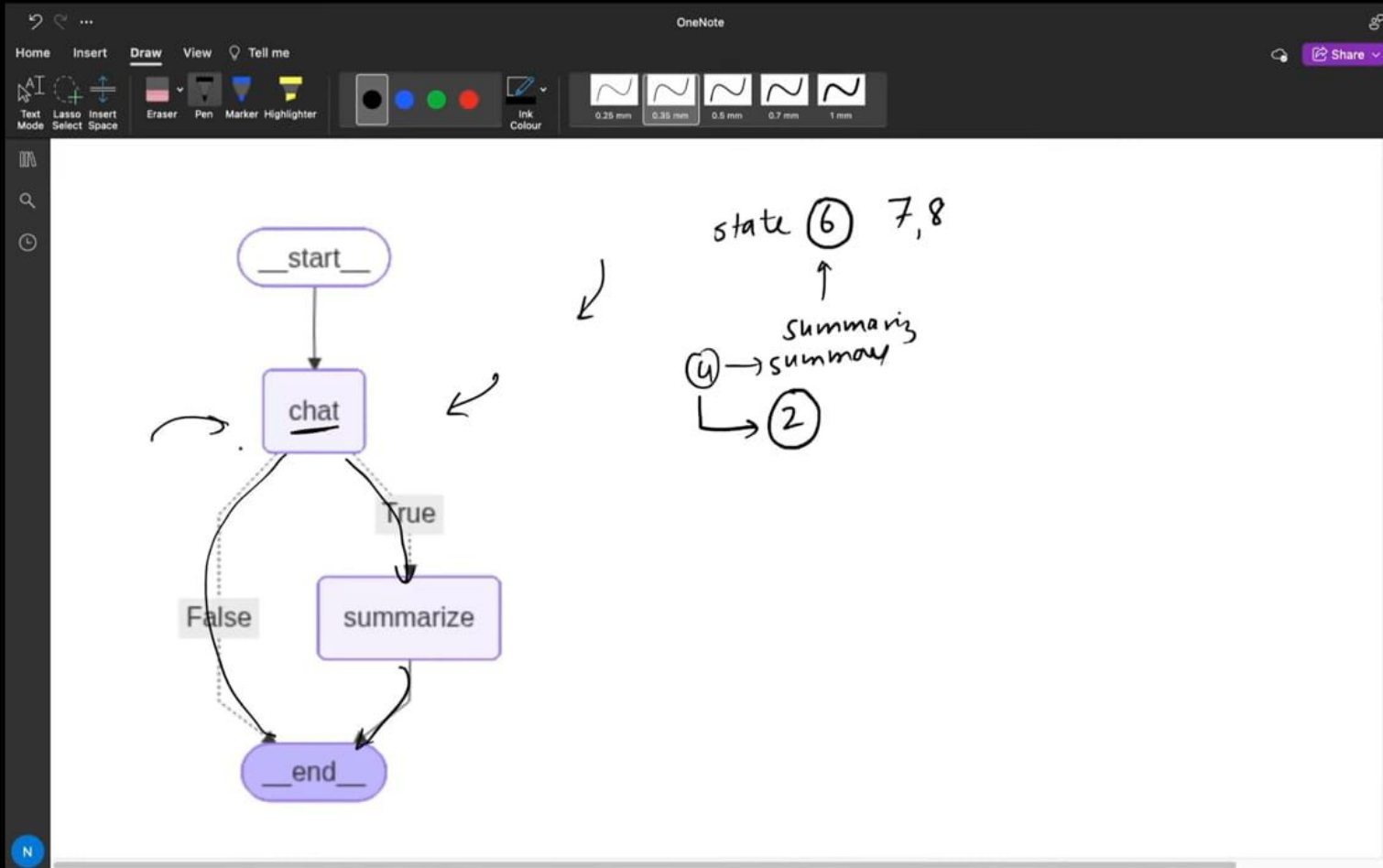
Context Overflow Problem

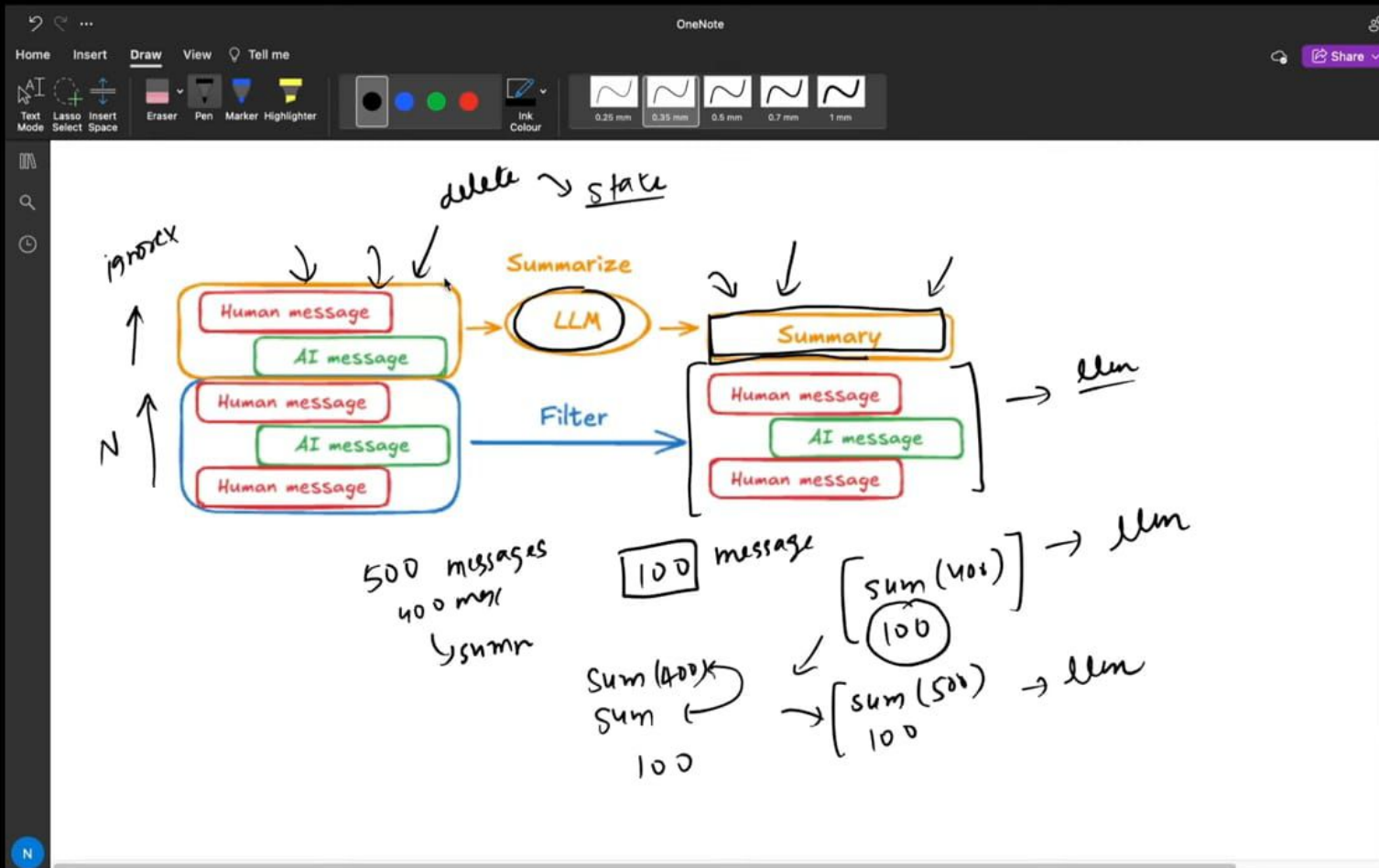
Sunday, 28 December 2025 12:17 AM

The diagram illustrates the Context Overflow Problem in a Large Language Model (LLM). It shows a sequence of messages (Human and AI) being processed. The context window is limited, and the total number of tokens (input and output) can exceed the maximum allowed, leading to a context overflow.

Handwritten notes and labels:

- LLM** (Large Language Model)
- Context window** (the limit on the number of tokens the model can process)
- input tokens** (tokens from the input message)
- max tokens** (the total number of tokens allowed in the context window)
- STM** (Short-Term Memory)
- long correlation** (referring to the long-term context or history)
- Human message** (input messages from the user)
- AI message** (output messages from the model)
- last N messages** (referring to the most recent messages in the context)
- max-tokens** (the total number of tokens allowed in the context window)
- 500 tokens** (a specific example of the token limit)
- 500** (a specific example of the token limit)
- messages** (the sequence of input and output messages)





OneNote

Home Insert Draw View Tell me

Text Lasso Insert Eraser Pen Marker Highlighter Ink Colour 0.35 mm 0.35 mm 0.5 mm 0.7 mm 1 mm

Long Term Memory

IntelliChat

TECH TALK

How does LangChain handle memory in autonomous agents?

LangChain uses 'ConversationBufferMemory' to store chat history and 'EntityMemory' for specific facts. It integrates with LLMs for context retention.

Can I use it with Python & OpenAI API?

Yes, simply install `pip install langchain openai`. Then, initialize the 'ChatOpenAI' model and a memory class. Here's a quick example:

```
from langchain.chat_models import ChatOpenAI
chat = ChatOpenAI(temperature=0)
```

Thanks! What about vector databases for retrieval?

It supports Pinecone, Chroma, and FAISS for semantic search and document retrieval. You can load data, create embeddings, and query efficiently.

TRAVEL PLANS

I'm planning a 2-week trip to Japan in October. Any suggestions for cities?

For a 2-week trip, I recommend splitting your time between Tokyo, Kyoto, and Osaka. They offer a mix of modern city life, tradition, and food culture.

What are the estimated costs for accommodation and food?

Accommodation ranges from \$80/night for hostels to \$200+/night for hotels. Food is diverse; street food is cheap (\$5-\$10), while restaurants average \$20-\$40 per meal. Budget around \$1500-\$2500 excluding flights.

Can you give a sample itinerary for the first few days in Tokyo?

Day 1: Arrive, check into Shinjuku hotel, explore Shibuya Crossing.
Day 2: Visit Senso-ji in Asakusa, Ueno Park museums, and Akihabara for electronics.
Day 3: Day trip to Nikko or Kamakura for shrines.

DEEP THOUGHTS

What does it mean to be truly conscious? Is AI conscious?

Consciousness is often defined as the state of being aware of and responsive to one's surroundings. Current AI lacks subjective experience or self-awareness; it processes data based on patterns, not understanding.

But if an AI can learn and adapt, isn't that a form of learning?

Machine learning allows AI to improve performance on tasks through data and aspects of learning. However, it does not possess genuine understanding or growth like biological beings.

How does reality relate to consciousness?

Reality is perceived through our senses and consciousness. A simulation is a constructed reality. The debate lies in whether a sufficiently advanced simulation could ever be indistinguishable from base reality to its inhabitants.

Type a message...

Type a message...

Type a message...

DB



OneNote

Home Insert Draw View Tell me

Text Mode Lasso Select Insert Space Eraser Pen Marker Highlighter Ink Colour 0.25 mm 0.35 mm 0.5 mm 0.7 mm 1 mm

base reality to its inhabitants.

Day 3: Day trip to Nikko or Kamakura for shrines.

LongGraph

store → BaseStore

create
search
edit
delete

In Memory store + RAM

Postgres store + Postgres

RedisStore



A man with a beard and mustache, wearing an orange hoodie, is speaking into a silver microphone. He is positioned in the bottom right corner of the OneNote window, appearing to be part of a video recording or live stream.